HOME ASSIGNMENT · BACKEND DEVELOPER

BookIt – Appointment Booking API

Thank you for the great conversation in the first interview! This home assignment is the next step in the process.

TIME BUDGET	STACK	BONUS SECTION
4–6 focused hours	Python 3.12 · Django 5 · DRF · PostgreSQL	Strictly optional

> **Please don't invest much more than 4–6 hours.** The Bonus section at the end is strictly optional – we'd rather see a smaller, clean, working solution than a large unfinished one.

The Story

> You're building the backend for **BookIt**, a platform where business owners (barbers, physiotherapists, tutors…) list their services, and clients book appointments with them.

Tech Requirements

- **Python 3.12 / Django 5** with **Django REST Framework**
- **PostgreSQL** as the database, accessed via the **Django ORM** (psycopg driver)
- Run Postgres locally with this one command (no local install needed):

```
docker run --name bookit-db -e POSTGRES_PASSWORD=bookit -e POSTGRES_DB=bookit \
  -p 5432:5432 -d postgres:16
```

- Use Django migrations (`makemigrations` / `migrate`) to create the schema, and seed the database on startup (e.g., a `manage.py` management command) with sample data: **~20 businesses, ~50 services, ~200 appointments, and a handful of users** (include the seeded login credentials in your README).

Data Model

ENTITY	FIELDS
User	Id, Email (unique), Password (hashed), FullName, Role (`Client` or `BusinessOwner`)
Business	Id, Name, Category, OwnerId → User
Service	Id, BusinessId → Business, Name, DurationMinutes, Price
Appointment	Id, ServiceId → Service, ClientId → User, StartTime, Status (`Pending` , `Confirmed` , `Cancelled`)

You may add fields if you need them. A custom user model is recommended.

Functional Requirements

1 Authentication (JWT)

- `POST /api/auth/register` – creates a user (email, password, full name, role).
- `POST /api/auth/login` – validates credentials and returns a **JWT** containing at least the user id and role as claims.
- **All other endpoints require a valid JWT.**
- Return the correct status codes: **401** when the caller isn't authenticated, **403** when they're authenticated but not allowed (see requirement 4).

2 Businesses – with Pagination

- `GET /api/businesses` – returns the list of businesses, **paginated**.
  - Support optional filtering by `category` and free-text `search` on the business name.
  - The response must include the **total count of matching businesses** (not just the current page).
  - How you design the pagination (parameters and response shape) is up to you.

3 Appointments – Filtering & Ownership

- `POST /api/appointments` – a `Client` books an appointment for a service.
- `GET /api/appointments?from=&to=&status=` – returns appointments **belonging to the caller only**, based on the JWT:
  - A `Client` sees their own appointments.
  - A `BusinessOwner` sees appointments for services of businesses they own.
  - Support optional filtering by date range and status.
- Return **explicit response serializers**, not raw model dumps, and include useful display fields (service name, business name) without causing an N+1 query problem.

4 Business Summary – ORM Aggregation

- `GET /api/businesses/{id}/summary` – only accessible to the **owner of that business** (anyone else gets 403). Returns, per service of the business:
  - the number of **upcoming** (future, non-cancelled) appointments
  - the total expected revenue from those appointments (count × price)
- Compute this **in the database** using Django ORM aggregation – not by looping over records in Python.

5 Middlewares (write at least 2 yourself)

Custom middleware classes – not just built-in framework settings:

1. **Global exception handling** – any unhandled exception returns a generic JSON error (e.g., Problem Details) with status 500. The client must **never** see stack traces or internal details; log the full exception server-side.
2. **Request logging** – logs method, path, response status code, and duration for every request, and attaches a **correlation ID** to the response headers.

6 Architecture & Layering

- Structure: Views → Service layer (e.g., `AppointmentService`) → Django ORM. Keep business logic out of views and serializers.
- Pass dependencies in **explicitly**, so the service layer can be tested without going through HTTP.
- Writes that must succeed or fail together run inside a **transaction** (`transaction.atomic`).

Deliverables

1. A Git repository (link or zip) with a **meaningful commit history** – commit as you go, not one big commit.
2. A `README.md` containing:
   - Setup and run instructions (including the Postgres docker command and seeded login credentials)
   - **A short written answer (2–3 sentences): how are database connections and transactions scoped per request in your solution, and why?**
3. Postman collection covering all endpoints.

Bonus (optional – only if you have time left)

- Refresh-token flow (short-lived access token + refresh endpoint)
- Unit tests for one service class (pytest)
- `docker-compose.yml` running both the API and Postgres
- Request validation with dedicated serializer validators (or Pydantic)
- Rate limiting on the login endpoint

Submission

Reply with the repository link (or zip). We'll schedule a **~45-minute session** to walk through your code together – be ready to explain your decisions.

Good luck – we're looking forward to seeing your work!